



UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO

LUCAS RODRIGUES ESTORCK PINTO
RAFAEL SARMENTO DE MENEZES

**MODELAGEM E DESENVOLVIMENTO DE UM SISTEMA DE SOFTWARE PARA
GERENCIAMENTO DE CLÍNICAS MÉDICAS**
*DA ESPECIFICAÇÃO DE REQUISITOS E DIAGRAMAS UML À IMPLEMENTAÇÃO DE
UM PRONTUÁRIO ELETRÔNICO COM ARQUITETURA MVC*

Nova Friburgo

2025

LUCAS RODRIGUES ESTORCK PINTO
RAFAEL SARMENTO DE MENEZES

**MODELAGEM E DESENVOLVIMENTO DE UM SISTEMA DE SOFTWARE PARA
GERENCIAMENTO DE CLÍNICAS MÉDICAS**

*DA ESPECIFICAÇÃO DE REQUISITOS E DIAGRAMAS UML À IMPLEMENTAÇÃO DE
UM PRONTUÁRIO ELETRÔNICO COM ARQUITETURA MVC*

Relatório de projeto apresentado à
Universidade do Estado do Rio de
Janeiro, como requisito parcial para a
disciplina de Sistemas de Bancos de
Dados do curso de Engenharia de
Computação.

Docente: Gustavo Libotte

NOVA FRIBURGO

2025

AGRADECIMENTOS

Agradecemos, primeiramente, a Deus, pela força e discernimento concedidos durante a jornada.

Ao nosso professor Gustavo Libotte, pela paciência e lições valiosas em aula.

Ao IPRJ, pela infraestrutura disponibilizada e pelo ambiente acadêmico que fomentou o desenvolvimento deste trabalho.

As famílias dos membros autores, por todo o apoio incondicional, incentivo e por proverem os meios necessários para nossa formação.

Qualquer tolo pode escrever um código que um computador entenda. Bons programadores escrevem códigos que os humanos possam entender.

- Martin Fowler

RESUMO

O presente trabalho detalha o projeto e a implementação de um Sistema de Gerenciamento de Clínica Médica (SIGM), concebido para otimizar o fluxo de atendimento ao paciente e a gestão administrativa. O objetivo central foi desenvolver uma aplicação web robusta que abrange desde o cadastro e autenticação de pacientes e médicos até à gestão completa do ciclo de vida das consultas. A metodologia adotada baseou-se em princípios de Engenharia de Software, iniciando com a especificação de requisitos através de Histórias de Usuário e a modelagem do sistema com Diagramas de Casos de Uso e Diagramas de Classes da UML. A implementação seguiu a arquitetura MVT (Model-View-Template), uma adaptação do padrão MVC, utilizando a linguagem Python e o framework Django. O sistema implementado oferece um prontuário eletrônico completo, permitindo ao médico consultar históricos, registrar anamneses e emitir receitas e atestados digitais. Notavelmente, o projeto inclui um módulo de pagamentos real, integrado via API com o gateway de pagamentos Stripe, processando transações de cartão de crédito de forma assíncrona através de *webhooks* para a confirmação automática das consultas. O resultado é um protótipo funcional que valida a eficácia da arquitetura e das tecnologias escolhidas para a criação de um sistema clínico seguro, modular e escalável.

Palavras-chave: Engenharia de Software. Django. Arquitetura MVT. Gestão de Clínicas. Prontuário Eletrônico. Stripe

ABSTRACT

This paper details the design and implementation of a Medical Clinic Management System (MCMS), conceived to optimize patient care workflows and administrative management. The central objective was to develop a robust web application covering everything from patient and doctor registration and authentication to the complete lifecycle management of appointments. The methodology adopted was based on Software Engineering principles, starting with requirements specification through User Stories and system modeling with UML Use Case and Class Diagrams. The implementation followed the MVT (Model-View-Template) architecture, an adaptation of the MVC pattern, utilizing the Python language and the Django framework. The implemented system offers a complete electronic health record (EHR), allowing physicians to consult histories, register anamneses, and issue digital prescriptions and medical certificates. Notably, the project includes a real payment module, integrated via API with the Stripe payment gateway, processing credit card transactions asynchronously through webhooks for automatic appointment confirmation. The result is a functional prototype that validates the effectiveness of the chosen architecture and technologies for creating a secure, modular, and scalable clinical system.

Keywords: Software Engineering. Django. MVT Architecture. Clinic Management. Electronic Health Record. Stripe.

AGRADECIMENTOS.....	3
RESUMO.....	5
ABSTRACT.....	6
1. INTRODUÇÃO.....	8
2. FUNDAMENTAÇÃO TEÓRICA.....	11
3. PROJETO E METODOLOGIA.....	16
4. IMPLEMENTAÇÃO E RESULTADOS.....	27
5. CONCLUSÃO.....	34

1. INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO E JUSTIFICATIVA

A gestão de informações em clínicas médicas é um pilar fundamental para a eficiência operacional e a qualidade do atendimento ao paciente. No entanto, muitos consultórios de pequeno e médio porte ainda dependem de processos manuais, agendamentos em papel ou sistemas de software legados, descentralizados e inseguros. Essa abordagem tradicional acarreta ineficiências significativas, como conflitos de agendamento, dificuldade no rastreamento de prontuários, falta de métricas financeiras e uma experiência fragmentada para o paciente.

O advento da transformação digital na saúde impõe a necessidade de sistemas centralizados, conhecidos como SIGM (Sistemas de Gerenciamento de Clínica Médica). Um sistema robusto não apenas automatiza o agendamento, mas também integra o Prontuário Eletrônico do Paciente (PEP), o faturamento e os canais de comunicação. A ausência de um sistema integrado gera gargalos: o médico perde tempo buscando históricos, a recepcionista gerencia manualmente os pagamentos e o paciente não tem autonomia para gerenciar suas próprias consultas ou acessar seus exames.

1.2 OBJETIVOS

Para entender a problemática apresentada, este projeto define um objetivo geral e um conjunto de objetivos específicos que guiaram o desenvolvimento.

1.2.1 Objetivo Geral

O objetivo geral deste trabalho é projetar, implementar e validar um protótipo funcional de um Sistema de Gerenciamento de Clínica Médica (SIGM), baseado em arquitetura web (MVT/MVC), capaz de gerenciar de forma integrada os perfis de usuários, o agendamento de consultas, o prontuário eletrônico e o processamento financeiro.

1.2.1 Objetivos Específicos

Para alcançar o objetivo geral, os seguintes objetivos específicos foram definidos:

1. Modelar os requisitos funcionais e não funcionais do sistema utilizando artefatos de Engenharia de Software, incluindo Histórias de Usuário, Diagramas de Casos de Uso (CDU) e Diagramas de Classes.
2. Implementar a arquitetura da solução utilizando a linguagem Python e o framework Django, traduzindo o modelo conceitual (Diagrama de Classes) para modelos de dados relacionais (`models.py`).
3. Desenvolver o módulo de autenticação e perfis, diferenciando as permissões e jornadas dos atores **Paciente**, **Médico** e **Recepcionista**.
4. Implementar o fluxo de agendamento, incluindo a lógica de verificação de conflitos (disponibilidade da agenda) e validação de regras de negócio.
5. Integrar um gateway de pagamento (Stripe) para o processamento de transações com cartão de crédito, implementando a lógica de *webhooks* para a confirmação assíncrona dos pagamentos.

6. Construir o módulo de Prontuário Eletrônico, permitindo ao médico registrar anamneses, emitir receitas e atestados vinculados a uma consulta específica.

1.3 ESTRUTURA DO TRABALHO

Este relatório está estruturado em cinco capítulos, visando apresentar a metodologia e os resultados de forma clara e sequencial.

O Capítulo 2 apresenta a fundamentação teórica, revisando os conceitos de Engenharia de Software (UML, GRASP), a arquitetura MVT do Django e as tecnologias de integração (APIs, Webhooks) utilizadas.

O Capítulo 3 detalha o projeto e a metodologia, apresentando os artefatos de requisitos (Histórias de Usuário) e modelagem (Diagramas). Esta seção também explicita as decisões de projeto e a tradução da modelagem conceitual para a arquitetura de implementação.

O Capítulo 4 descreve a implementação e os resultados obtidos, detalhando o desenvolvimento de cada módulo funcional (Paciente, Médico, Pagamentos) e validando os fluxos com capturas de tela.

O Capítulo 5 conclui o trabalho, apresentando os resultados alcançados em relação aos objetivos propostos e sugerindo trabalhos futuros para a evolução do sistema.

2. FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta a base conceitual sobre a qual o presente projeto foi estruturado. São revisados os princípios de Engenharia de Software e modelagem UML (Unified Modeling Language), os padrões de arquitetura e projeto que guiaram a implementação, e as tecnologias-chave que viabilizaram as funcionalidades do sistema.

2.1 ENGENHARIA DE SOFTWARE E MODELAGEM UML

A Engenharia de Software é uma disciplina da engenharia dedicada ao desenvolvimento de software, aplicando uma abordagem sistemática, disciplinada e quantificável ao projeto, desenvolvimento, operação e manutenção de sistemas. Para gerenciar a complexidade inerente ao desenvolvimento, esta disciplina emprega técnicas de modelagem, sendo a UML (Unified Modeling Language) o padrão *de facto* da indústria. A UML fornece uma notação gráfica para especificar, visualizar, construir e documentar os artefatos de um sistema de software.

2.1.1 Histórias de Usuário e Critérios de Aceitação

No contexto de metodologias ágeis de desenvolvimento, o levantamento de requisitos é frequentemente realizado através de Histórias de Usuário. Uma História de Usuário é uma descrição concisa de uma funcionalidade, narrada sob a perspectiva do ator que a deseja. Sua estrutura clássica é: "Como [ator], eu quero [ação/funcionalidade], para que [objetivo/valor]".

Esta técnica foca no valor entregue ao usuário final, em vez de em especificações técnicas prescritivas. Para garantir a estabilidade e a completude, cada História de Usuário é associada a Critérios de Aceitação (CA). Os CAs definem os limites da funcionalidade e servem como um contrato entre o desenvolvedor e o cliente, especificando as condições que o software deve atender para que a história seja considerada "concluída"

2.1.2 Diagrama de Casos de Uso (CDU)

O Diagrama de Casos de Uso é um artefato UML focado em capturar os requisitos funcionais e o comportamento *externo* do sistema. Ele ilustra as interações entre os atores (entidades externas, como **Paciente** ou **Médico**) e os casos de uso (as funcionalidades que o sistema oferece, como **Marcar Consulta** ou **Pagar Consulta Online**). Este diagrama é fundamental na fase de análise, pois delimita o escopo do sistema e define suas fronteiras.

2.1.3 Diagrama de Classes

O Diagrama de Classes descreve a estrutura *estática* interna de um sistema. Ele é o principal *blueprint* para a construção do banco de dados e do domínio lógico. Este diagrama modela as classes do sistema (como **USUARIO**, **CONSULTA**), seus atributos (como **data**, **hora**, **status**) e os relacionamentos entre elas (como associação, agregação e herança). A cardinalidade (ex: 1 para 1, 1 para N) especificada nos relacionamentos é crucial para definir as regras de negócio.

2.2 ARQUITETURA DE SOFTWARE E PADRÕES DE PROJETO

A arquitetura de software define a estrutura de alto nível do sistema, os seus componentes e as interações entre eles. A escolha de um padrão

arquitetural adequado é vital para garantir atributos de qualidade como manutenibilidade, escalabilidade e testabilidade.

2.2.1 O Padrão Model-View-Controller (MVC)

O MVC é um padrão de arquitetura canônico para sistemas interativos. Ele propõe a separação do sistema em três componentes interligados:

- **Model (Modelo):** Encapsula os dados e a lógica de negócios. É o "cérebro" do sistema, responsável pela validação dos dados e pela persistência no banco de dados.
- **View (Visão):** Responsável pela apresentação dos dados ao usuário, ou seja, a interface do usuário (UI).
- **Controller (Controlador):** Atua como o intermediário. Ele recebe as entradas do usuário (vindas da *View*), processa-as (invocando o *Model*) e atualiza a *View* com os resultados.

2.2.2 A Arquitetura Model-View-Template (MVT) do Django

O framework Django implementa uma variação do MVC, conhecida como MVT (Model-View-Template). A correspondência de responsabilidades é ligeiramente diferente:

- **Model (Modelo):** Idêntico ao MVC, representa os dados e a lógica (`models.py`).
- **View (Visão):** No Django, a **View** (`views.py`) atua como o Controlador do MVC. Ela contém a lógica que recebe requisições HTTP, interage com o *Model* e decide qual resposta enviar.

- Template (Modelo de Apresentação): É o componente que de fato corresponde à Visão do MVC. É o arquivo (ex: `.html`) que define a estrutura da apresentação, sendo renderizado pela *View*.

Esta arquitetura MVT foi a escolhida para o projeto, alinhando-se com os artefatos UML, onde os `Controllers` do diagrama foram implementados como `Views` em Python.

2.2.3 Padrões GRASP (General Responsibility Assignment Software Patterns)

Os padrões GRASP são um conjunto de heurísticas e princípios utilizados para atribuir responsabilidades às classes em um projeto orientado a objetos, visando baixa acoplamento e alta coesão. No presente projeto, três padrões foram centrais:

- Information Expert (Especialista da Informação): Este princípio dita que a responsabilidade deve ser atribuída à classe que possui a informação necessária para cumpri-la. No nosso sistema, em vez de um "gerenciador" externo, a própria classe `Consulta` é a especialista em calcular seu status, e o `ConsultaForm` é o especialista em validar um agendamento futuro.
- Controller (Controlador): O GRASP define um *Controller* como o objeto que recebe e coordena eventos do sistema, servindo como uma fachada entre a interface (Template) e a lógica de domínio (Model). Esta responsabilidade foi atribuída às classes do `views.py` (ex: `MarcarConsultaView`, `AnamneseView`), que recebem as requisições HTTP e orquestram a lógica de negócio.
- Creator (Criador): Este padrão sugere que a classe B deve ser responsável por criar instâncias da classe A se B "contém" ou "agrega" A, ou se B "usa" A de perto. No nosso projeto, o `ConsultaForm` é o

Criador das instâncias de `Consulta`, pois ele agrega os dados necessários e valida sua criação.

2.3 TECNOLOGIAS E FERRAMENTAS

2.3.1 Python e o Framework Django

Python foi a linguagem de programação escolhida por sua sintaxe clara e vasto ecossistema. O Django, um framework web *high-level* escrito em Python, foi selecionado por sua filosofia "baterias inclusas". O componente central utilizado foi o seu ORM (Object-Relational Mapper), que abstrai o banco de dados (SQLite, neste protótipo) e permite a tradução direta do Diagrama de Classes UML para classes Python (`models.py`), garantindo a integridade dos dados e acelerando o desenvolvimento.

2.3.2 APIs de Pagamento e Webhooks(Stripe)

Para atender ao requisito de "Pagar Consulta Online", foi necessária a integração com um gateway de pagamento. Uma API (Application Programming Interface), neste contexto, é o contrato de comunicação que permite ao nosso servidor interagir com um sistema externo (Stripe).

Dado que o processamento de pagamentos (especialmente com cartão ou PIX) é uma operação assíncrona, o sistema não pode simplesmente "esperar" pela confirmação. A solução para este problema é o uso de Webhooks. Um Webhook é um mecanismo de *callback* reativo baseado em HTTP. O fluxo é o seguinte:

1. Nosso sistema (via `CriarCheckoutSessionView`) envia o usuário ao Stripe.

2. O usuário paga no ambiente do Stripe.
3. O Stripe, ao confirmar o pagamento, envia uma notificação (um **HTTP POST**) para uma URL pré-configurada em nosso servidor (o *endpoint* do Webhook).
4. Nossa `stripe_webhook_view` recebe essa notificação, valida sua autenticidade (via assinatura criptográfica do cabeçalho) e, só então, atualiza o status da **Consulta** no banco de dados para **PAGA**.

Esta arquitetura assíncrona desacopla os sistemas e é a única forma robusta de lidar com confirmações de pagamento externas.

3. PROJETO E METODOLOGIA

Este capítulo detalha o processo de engenharia adotado para traduzir os requisitos de negócio em uma solução de software funcional. A metodologia de desenvolvimento foi fundamentada na especificação formal de requisitos através de Histórias de Usuário e na modelagem visual do sistema com artefatos UML.

A seção subsequente apresenta estes artefatos de projeto — que serviram como contrato e guia para a implementação — e, mais importante, articula as decisões de arquitetura e projeto tomadas para traduzir esses modelos conceituais para a arquitetura Model-View-Template (MVT) do framework Django.

3.1 LEVANTAMENTO DE REQUISITOS (HISTÓRIAS DE USUÁRIO)

O levantamento de requisitos funcionais foi conduzido através da técnica de Histórias de Usuário (User Stories), conforme detalhado no Capítulo 2. Esta abordagem permitiu decompor as complexas necessidades do sistema em unidades de funcionalidade discretas e testáveis, cada uma focada no valor agregado para um ator específico.

As Histórias de Usuário e seus respectivos Critérios de Aceitação (CA) foram categorizados pelos quatro atores identificados no sistema: **Paciente**,

Recepcionista, Médico e Sistema. As tabelas a seguir apresentam o conjunto completo de requisitos que nortearam o desenvolvimento.

ID	História de Usuário (Paciente)	Crítérios de Aceitação (CA)
PAC-01	Como paciente, quero me cadastrar no sistema, para criar meu perfil e acessar serviços online.	1. Formulário com nome, CPF, data de nascimento, telefone, e-mail. 2. Validação de CPF e e-mail. 3. Confirmação por e-mail ou SMS. 4. Conta criada e usuário redirecionado para dashboard.
PAC-02	Como paciente, quero fazer login com e-mail/CPF e senha, para acessar meu perfil e marcar consultas.	1. Autenticação segura. 2. Recuperação de senha por e-mail ou SMS. 3. Bloqueio após tentativas suspeitas. 4. Sessão expira após inatividade.
PAC-03	Como paciente, quero marcar consulta com médico disponível, para agendar atendimento no horário conveniente.	1. Escolher especialidade, médico, data e horário com verificação de conflitos. 2. Confirmação imediata e notificação por e-mail/SMS. 3. Exibir tempo estimado de espera.
PAC-04	Como paciente, quero cancelar minha consulta, para liberar o horário caso eu não possa comparecer.	1. Opção de cancelar até prazo configurável (ex.: 24h antes). 2. Política de cancelamento exibida. 3. Confirmação de cancelamento enviada. 4. Vaga liberada na agenda do médico.
PAC-05	Como paciente, quero consultar meus exames e resultados, para acompanhar meu histórico clínico.	1. Listar exames por data e tipo. 2. Visualização de arquivos (PDF/imagem) com controle de acesso. 3. Data do laudo e profissional responsável exibidos. 4. Possibilidade de download seguro.
PAC-06	Como paciente, quero pagar a consulta online, para garantir atendimento sem necessidade de pagamento presencial.	1. Opção de pagamento via cartão ou gateway configurado. 2. Geração de comprovante/recibo. 3. Atualização automática do status da consulta como "paga". 4. Tratamento de falha de pagamento.

ID	História de Usuário (Recepcionista)	Crerios de Aceitação (CA)
REC-01	Como recepcionista, quero emitir comprovante de atendimento/pagamento, para entregar registro oficial ao paciente.	1. Gerar comprovante com dados do paciente, serviço, valor, data, meio de pagamento e assinatura digital (quando aplicável). 2. Possibilidade de impressão e envio por e-mail.
REC-02	Como recepcionista, quero atualizar a agenda médica (incluir/alterar horários, bloquear períodos), para refletir mudanças e ausências dos médicos.	1. Editar disponibilidade do médico. 2. Bloquear horários por férias/plantões. 3. Notificações enviadas automaticamente a pacientes afetados. 4. Auditoria de alterações com usuário e timestamp.
REC-03	Como recepcionista, quero consultar a lista de pacientes do dia e seus pagamentos pendentes, para organizar fluxo e cobrança.	1. Filtro por data, médico e status. 2. Indicações visuais de pendências. 3. Links rápidos para registrar pagamento ou remarcar consultas.

ID	História de Usuário (Médico)	CrITÉrios de Aceitação (CA)
MED-01	Como médico, quero consultar minha agenda diária/semanal, para ver rapidamente meus pacientes e horários.	1. Visualização por dia/semana/mês. 2. Detalhes do paciente (nome, idade, motivo da consulta). 3. Sinalização de consultas pagas/pendentes.
MED-02	Como médico, quero emitir receita médica digital, para fornecer medicação prescrita de forma segura e legível.	1. Formulário com medicamentos, posologia e duração. 2. Assinatura digital do médico. 3. Geração de PDF com QR code para verificação. 4. Registro ligado ao prontuário do paciente.
MED-03	Como médico, quero emitir atestado médico, para justificar faltas ou afastamentos do paciente.	1. Preencher dados necessários (período, CID opcional conforme regra). 2. Assinatura digital. 3. Geração de documento imprimível. 4. Registro no histórico do paciente.

ID	História de Usuário (Sistema)	CrITÉrios de Aceitação (CA)
SIS-01	Como sistema, quero coletar avaliações de satisfação pós-atendimento, para medir qualidade e apontar melhorias.	1. Disparo automático de pesquisa após consulta (por e-mail/SMS). 2. Perguntas objetivas e campo para comentário. 3. Armazenamento e painel com métricas agregadas.
SIS-02	Como sistema, quero enviar lembrete automático de consultas, para reduzir faltas e melhorar a adesão.	1. Agendamento de lembretes configuráveis (ex.: 72h, 24h, 2h antes). 2. Envio por e-mail/SMS/push. 3. Cancelamento automático se a consulta for cancelada. 4. Registro de entregas e falhas.
SIS-03	Como sistema, quero gerar relatórios financeiros mensais para impressão, para apoiar a gestão e contabilidade da clínica.	1. Consolidar receitas por fonte (consultas, exames), despesas vinculadas, impostos e saldo. 2. Filtros por médico, centro de custo e período. 3. Exportação PDF para impressão.

3.2 MODELAGEM DO SISTEMA (ARTEFATOS UML)

Para traduzir os requisitos em uma estrutura de software, foram utilizados artefatos da Unified Modeling Language (UML), que fornecem uma notação gráfica padronizada para a modelagem do sistema.

3.2.1 Diagrama de Casos de Uso (CDU)

O Diagrama de Casos de Uso (CDU) foi empregado para delimitar o escopo do sistema e visualizar as interações de alto nível entre os atores e as funcionalidades principais. O diagrama define as fronteiras do "Sistema Clínica Médica" e identifica as responsabilidades de cada ator.

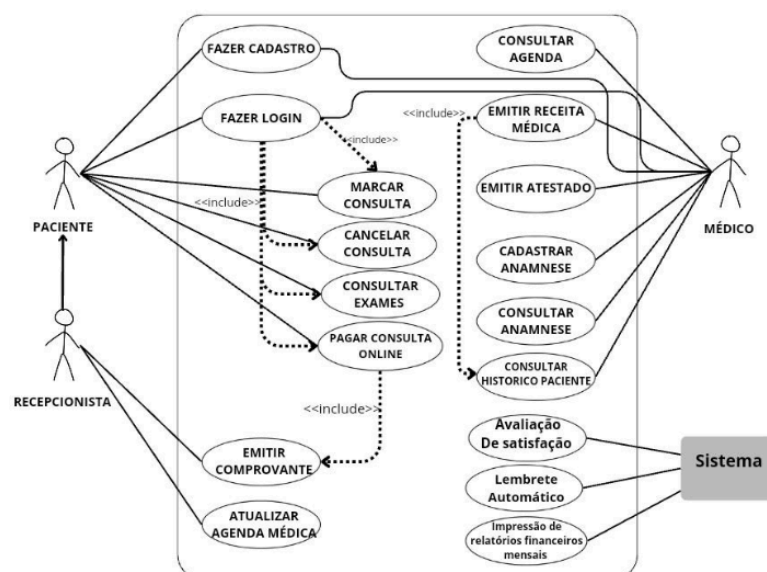
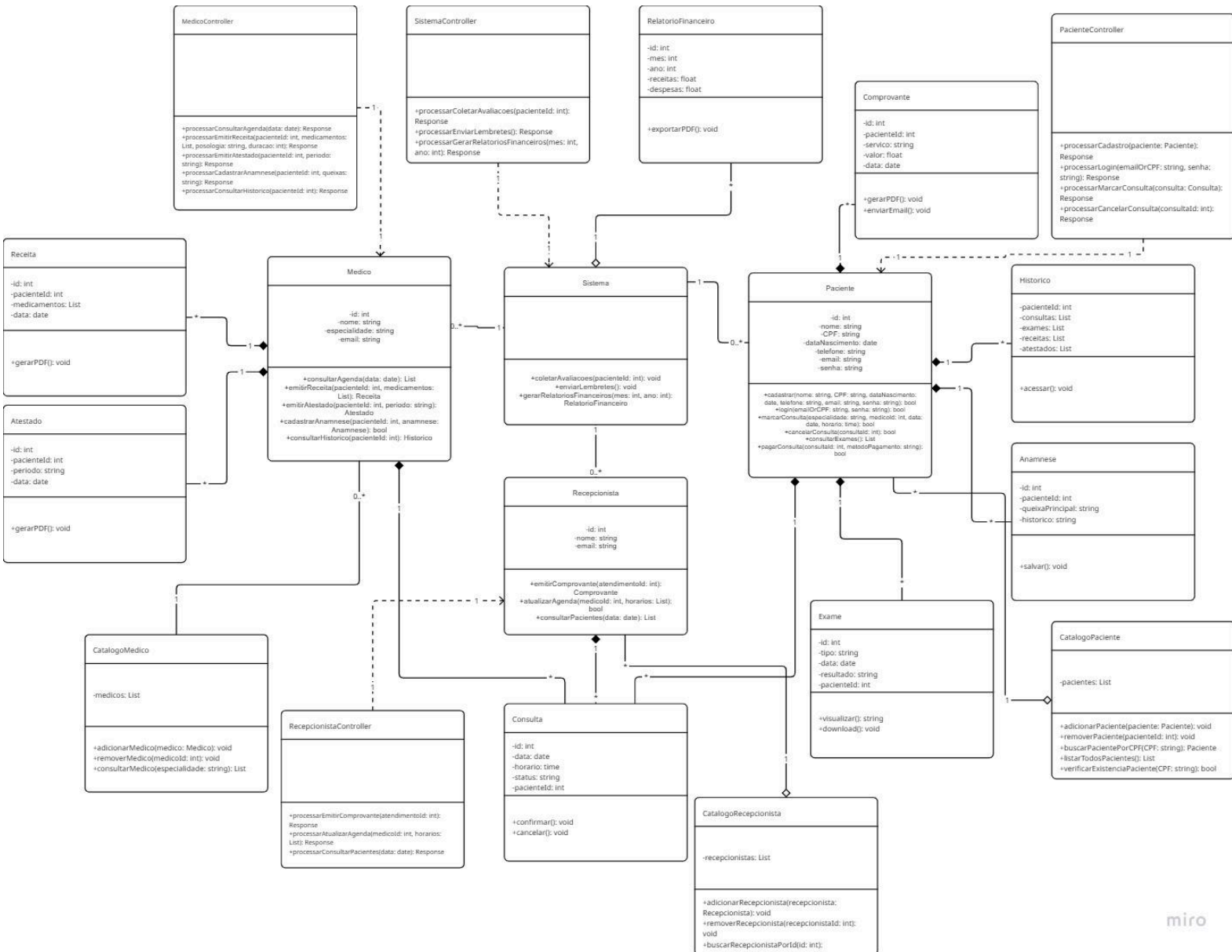


Figura 1: Diagrama de casos de uso do sistema (Fonte: Autores)



3.2.2 Diagrama de Classes

Figura 2: Diagrama de classes do sistema (Fonte: Autores)

O Diagrama de Classes foi o artefato central para o projeto da arquitetura de dados e da lógica de negócios. Ele descreve a estrutura estática do sistema, incluindo as classes de domínio (**USUARIO**, **CONSULTA**, **PRONTUARIO**), seus atributos, e os relacionamentos (associação, agregação e herança) entre elas. Este diagrama serviu como o *blueprint* direto para a implementação dos Modelos no framework Django.

3.3 ARQUITETURA DA SOLUÇÃO E DECISÕES DE PROJETO

A fase de implementação exigiu a tradução dos modelos conceituais (UML) para a arquitetura concreta MVT (Model-View-Template) do Django. Esta seção detalha as decisões de engenharia de software tomadas durante esse processo.

3.3.1 Tradução do Diagrama de Classes para Modelos Django

(models.py)

O Diagrama de Classes apresentava um desafio de implementação na hierarquia de **USUARIO**. Uma implementação literal da herança (**USUARIO** como classe-pai, **PACIENTE**, **MEDICO** e **RECEPCIONISTA** como classes-filhas) resultaria em múltiplas tabelas e junções complexas (joins) no banco de dados.

Decisão de Projeto: Optou-se por uma abordagem mais performática e alinhada às melhores práticas do Django. Foi utilizado o modelo **AbstractUser** nativo do framework para a base de autenticação, estendendo-o para criar um **CustomUser**. A diferenciação dos atores (**PACIENTE**, **MEDICO**, **RECEPCIONISTA**) foi implementada através de um único campo **tipo_usuario** (do tipo **TextChoices**) dentro do **CustomUser**. Esta decisão simplifica as consultas, mantém a

integridade em uma única tabela de usuários e utiliza o robusto sistema de permissões do Django.

A classe **PRONTUARIO** foi implementada como um conjunto de modelos discretos (**Anamnese**, **Receita**, **Atestado**, **Exame**), cada um mantendo uma **ForeignKey** (chave estrangeira) para o modelo **Consulta** ou **CustomUser** (Paciente), refletindo a agregação vista no diagrama.

3.3.2 Tradução dos Controladores UML para o Padrão MVT (Views.py)

O Diagrama de Classes especifica classes de controle como **PacienteController** e **MedicoController**, com métodos como **MarcarConsulta()** e **CadastrarAnamnese()**.

Decisão de Projeto: No padrão MVT, a responsabilidade do *Controller* (conforme o padrão GRASP) é atribuída às **Views** (definidas em **views.py**). Portanto, em vez de uma única classe monolítica **PacienteController**, foi implementado um conjunto de *views* baseadas em classe (Class-Based Views), cada uma tratando uma responsabilidade única. Por exemplo:

- O método **MarcarConsulta()** foi implementado como a classe **MarcarConsultaView(View)**.
- O método **CadastrarAnamnese()** foi implementado como a classe **AnamneseView(View)**.

Esta abordagem adere ao Princípio da Responsabilidade Única (SRP) e mantém o código coeso e desacoplado.

3.3.3 Implementação de Classes Conceituais (Agenda) como Lógica de Negócio

O Diagrama de Classes inclui uma classe conceitual **AGENDA**, com o método **verificarDisponibilidade(dataHora)**.

Decisão de Projeto: Uma análise técnica concluiu que a **AGENDA** não representa uma entidade de dados (um substantivo), mas sim um *processo* (um verbo). Criar um modelo **Agenda** no banco de dados seria redundante, pois toda a informação necessária já existe no modelo **Consulta**.

Portanto, a "agenda" foi implementada como uma *lógica de negócio* dentro do *controller* (**MarcarConsultaView**). O método **verificarDisponibilidade()** foi traduzido para uma *query* (consulta) no banco de dados, que verifica a existência de conflitos no modelo **Consulta** para um determinado médico e horário. Esta decisão evita a redundância de dados e torna o sistema mais eficiente.

3.3.4 Refatoração do Sistema de Autenticação (EmailOrUsernameBackend)

Seguindo as Histórias de Usuário, o requisito inicial incluía um **username** para login. Contudo, uma reanálise de usabilidade concluiu que o padrão web moderno utiliza o e-mail como identificador primário.

Decisão de Projeto: O sistema foi refatorado para remover o campo **username** do formulário de cadastro do paciente. Para manter a flexibilidade (permitindo que o **admin** e o **Medico** ainda pudessem usar um *username*), foi implementado um *backend* de autenticação customizado, o **EmailOrUsernameBackend**. Esta classe intercepta a tentativa de login e permite a autenticação verificando o *input* do usuário tanto contra o campo **username** quanto contra o campo **email** no banco de dados, através de uma consulta **Q (OR)**.

3.4 MODELAGEM FÍSICA E NORMALIZAÇÃO DO BANCO DE DADOS

O projeto do banco de dados foi concebido visando o equilíbrio entre a integridade dos dados e o desempenho. O esquema relacional resultante (implementado via SQLite) encontra-se majoritariamente na Terceira Forma Normal (3NF).

- Estratégia de Herança (Single Table): Optou-se por implementar **Paciente**, **Médico** e **Recepcionista** em uma única tabela física (**core_customuser**), diferenciados por um discriminador (**tipo_usuario**). Isso evita *JOINS* desnecessários durante a autenticação.
- Normalização do Prontuário: Documentos como **Receita** e **Atestado** foram separados em tabelas próprias (**core_receita**, **core_atestado**), vinculadas à **core_consulta** por Chaves Estrangeiras (*Foreign Keys*). Isso evita campos nulos na tabela de consulta e garante a integridade referencial (1:N).
- Deleção Lógica: Para preservar o histórico financeiro, registros não são excluídos fisicamente (*Hard Delete*). Utiliza-se um atributo de estado (**status='CANCELADA'**) na tabela **core_consulta**.

4. IMPLEMENTAÇÃO E RESULTADOS

Este capítulo apresenta os resultados práticos obtidos através da tradução dos artefatos de projeto (Capítulo 3) em um software funcional. A implementação seguiu a arquitetura MVT (Model-View-Template) do framework Django, e as seções a seguir detalham o desenvolvimento e a validação de cada módulo principal do sistema, demonstrando o cumprimento dos requisitos e a aplicação das decisões de engenharia.

4.1 MÓDULO DE AUTENTICAÇÃO E PERFIS

A fundação do sistema é o seu módulo de autenticação e gerenciamento de perfis, que implementa a hierarquia de **USUARIO** definida no Diagrama de Classes. Conforme a decisão de projeto (Seção 3.3.1), foi utilizado o **AbstractUser** do Django, estendido para um **CustomUser** contendo o campo **tipo_usuario**.

A principal refatoração neste módulo foi a remoção da dependência do **username** para o **Paciente**, alinhando o sistema às práticas web modernas.

- Implementação (Cadastro): A classe **PacienteCreationForm** foi modificada para omitir o campo **username**. No método **save()** do formulário, o **username** do usuário é preenchido programaticamente com o valor do campo **email**.
- Implementação (Login): Para permitir que o **Paciente** (ou **Medico**) fizesse login com seu e-mail, enquanto o **admin** poderia usar seu **username**, foi implementado um *backend* de autenticação customizado, o **EmailOrUsernameBackend**. Conforme detalhado na Seção 3.3.4, este *backend* utiliza um objeto **Q** do ORM do Django para consultar o banco de dados, permitindo a autenticação (**WHERE username=input OR email=input**).

O resultado é um fluxo de cadastro mais limpo para o paciente e um login mais flexível para todos os atores.

4.2 MÓDULO DO PACIENTE (FLUXO DE ATENDIMENTO)

Este módulo implementa a jornada completa do ator **Paciente**, desde o agendamento até a consulta de seus registros.

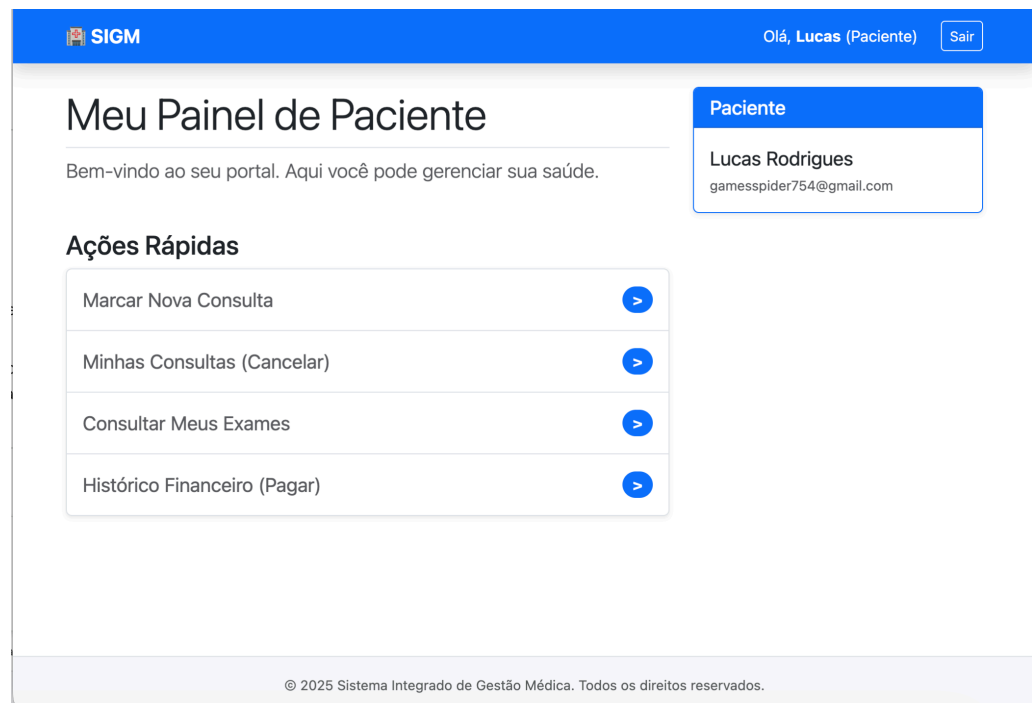


Figura 3: Interface de usuário (UI) do painel home do módulo de paciente (Fonte: Autores)

4.2.1 Cadastro, Agendamento e Cancelamento de Consultas

Para implementar o Caso de Uso "Marcar Consulta", foi criada a **MarcarConsultaView**. A lógica de negócio "verificarDisponibilidade", definida na classe conceitual **AGENDA**, foi implementada nesta *view* através de consultas ao ORM. O método

`post` da `view` executa duas verificações de conflito (`.exists()`) antes de salvar a nova instância de `Consulta`:

1. Verifica se o `Medico` selecionado já possui uma consulta naquele `data_hora`.
2. Verifica se o `Paciente` (o `request.user`) já possui outra consulta naquele `data_hora`.

Figura 4: *UI* de consultas (Fonte: Autores)

A validação de datas passadas foi delegada ao `ConsultaForm`, através do método `clean_data_hora()`, garantindo a coesão do formulário.

A funcionalidade "Cancelar Consulta" foi implementada na `CancelarConsultaView`. Esta `view` aplica a regra de negócio do Critério de Aceitação, comparando `timezone.now()` com o `consulta.data_hora - timedelta(hours=24)`. Apenas requisições `POST` são aceitas para esta ação, garantindo que uma simples navegação (`GET`) não possa modificar o estado dos dados.

4.2.2 Módulo de Pagamentos (Integração Stripe)

O requisito "Pagar Consulta Online" foi implementado utilizando a API do gateway Stripe. A classe `CriarCheckoutSessionView` atua como o *controller* (GRASP) que coordena a transação.

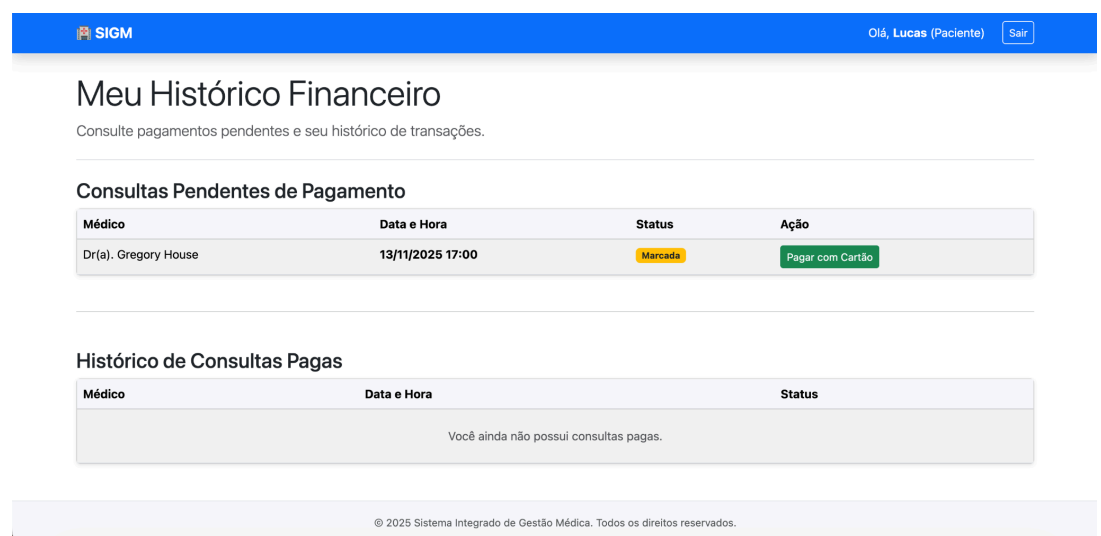


Figura 5: Painel de pagamentos do usuário paciente (Fonte: Autores)

Ao ser invocada (via `POST` pelo paciente), esta *view*:

1. Instancia o cliente Stripe com a chave secreta (`settings.STRIPE_SECRET_KEY`).
2. Chama o método `stripe.checkout.Session.create()`, passando os dados da consulta (valor, descrição) e os métodos de pagamento (ex: `card`).
3. Define as `success_url` e `cancel_url`, que são *endpoints* locais para onde o Stripe deve redirecionar o usuário.
4. Decisão Crítica: Antes de redirecionar o usuário, o ID da sessão de checkout (`checkout_session.id`, prefixo `cs_...`) é salvo no

campo `stripe_checkout_id` da *Consulta*. Este passo é a chave para a rastreabilidade do *webhook*.

5. O usuário é redirecionado para a URL de pagamento segura do Stripe (`checkout_session.url`).

4.2.3 Processamento Assíncrono de Pagamentos via Webhook

Esta é a implementação mais complexa do sistema, tratando da natureza assíncrona dos pagamentos. O fluxo garante que o sistema não dependa do redirecionamento do usuário (que pode fechar o navegador) para confirmar o pagamento.

1. Endpoint: Foi criada a `stripe_webhook_view` e a rota `/pagamentos/stripe-webhook/`. Esta *view* é isenta de proteção CSRF (`@csrf_exempt`), pois deve receber requisições **POST** diretamente do servidor do Stripe.
2. Segurança: A *view* utiliza a `HTTP_STRIPE_SIGNATURE` do cabeçalho da requisição e a chave secreta do *webhook* (`settings.STRIPE_WEBHOOK_SECRET`) para construir e verificar o evento (`stripe.Webhook.construct_event`). Isso garante que a requisição é autêntica e não foi forjada.
3. Lógica: A *view* escuta pelo evento `checkout.session.completed`. Ao recebê-lo, ela extrai o `checkout_session_id` (`session.get('id')`).
4. Atualização: O sistema busca no banco de dados a *Consulta* que possui aquele `stripe_checkout_id`

(`Consulta.objects.get(stripe_checkout_id=...)`) e, ao encontrá-la, atualiza seu `status` para `PAGA`.

5. Resposta: A *view* retorna um `HttpResponse(status=200) OK` ao Stripe, confirmando o recebimento bem-sucedido do aviso.

4.2.4 Consulta de Exames e Upload de Mídia

Para implementar a "Consulta de Exames", o Django foi configurado para gerenciar o upload de arquivos de mídia (PDFs, imagens). Os `settings.py` foram atualizados com `MEDIA_URL` e `MEDIA_ROOT`, e o `urls.py` principal foi modificado para servir esses arquivos em modo de desenvolvimento. O modelo `Exame` utiliza um `FileField`, que gerencia o upload do arquivo para o diretório especificado. A `MinhasExamesView` simplesmente consulta os exames do paciente (`Exame.objects.filter(paciente=request.user)`) e o template renderiza o link (`{{ exame.arquivo_resultado.url }}`) para o download.

4.3 MÓDULO DO MÉDICO (FLUXO DE PRONTUÁRIO)

Este módulo implementa a jornada do ator `Médico`, focada na gestão da agenda e no registro de informações clínicas.

4.3.1 Consulta de Agenda e Prontuário do Paciente

A `HomeView` do sistema atua como um roteador de alto nível. Ela verifica o `tipo_usuario` do `request.user` logado. Se o tipo for `MEDICO`, a *view* coleta as consultas associadas a ele, separando-as em `consultas_hoje` e `consultas_futuras`, e renderiza o template `medico_home.html`.

A funcionalidade "Consultar Histórico Paciente" foi implementada na `ProntuarioView`. Na agenda do médico, o nome de cada paciente é um hiperlink para a rota `prontuario_paciente`, passando o `paciente_id`. A `ProntuarioView` (Figura 5) recebe este ID, busca o `CustomUser` (paciente) e, em seguida, executa múltiplas consultas ao banco para agregar *todos* os artefatos relacionados a ele: `Consulta.objects.filter(paciente=paciente)`, `Exame.objects.filter(paciente=paciente)`, etc. O template `prontuario.html` então renderiza esse histórico cronológico.

4.3.2 Emissão de Documentos (Anamnese, Receita e Atestado)

O fluxo de "Cadastrar Anamnese" e emitir documentos foi centralizado em um "Painel de Atendimento" (o template `anamnese_form.html`).

- Implementação: As `views` `AnamneseView`, `ReceitaView` e `AtestadoView` seguem o mesmo padrão lógico: elas recebem um `consulta_id` pela URL e implementam uma lógica "get-or-create". O método `get` tenta buscar um objeto existente (`Anamnese.objects.get(consulta=...)`) e, se falhar (`DoesNotExist`), instancia um formulário vazio. O método `post` faz o mesmo, passando o `request.POST` para criar ou atualizar o registro.
- Resultados: A página de Anamnese serve como um *hub*, exibindo o formulário de anamnese e links diretos (passando o `consulta_id`) para as `views` de Receita e Atestado, criando um fluxo de trabalho coeso para o médico.

4.3.3 Conclusão do Atendimento

Para finalizar o ciclo de vida da consulta, foi criada a `ConcluirConsultaView`. Esta `view` (que aceita apenas `POST` por

segurança) é invocada pelo botão "Encerrar Atendimento" no Painel de Atendimento. Sua única responsabilidade é buscar a **Consulta** pelo **consulta_id** e alterar seu **status** para **CONCLUIDA**, retirando-a da agenda futura do médico.

4.4 MÓDULO ADMINISTRATIVO (RECEPCIONISTA E RELATÓRIOS)

Para atender aos requisitos do ator Recepcionista, foi implementado um painel de controle exclusivo (**recepcionista_home.html**). Diferente do painel médico, este *dashboard* oferece uma visão global da clínica, listando a agenda consolidada de todos os médicos para o dia corrente.

Adicionalmente, foi desenvolvido o módulo de inteligência de negócios solicitado: a Geração de Relatórios Financeiros. Através da **RelatorioFinanceiroView**, o sistema agrega todas as consultas com status **PAGA** ou **CONCLUÍDA** do mês atual, calcula o faturamento total e apresenta os dados em uma interface tabular pronta para impressão, permitindo o controle de caixa da clínica.

5. CONCLUSÃO

Este capítulo finaliza o relatório do projeto, apresentando uma análise dos resultados obtidos em contraste com os objetivos estabelecidos na introdução. Adicionalmente, são discutidas as limitações do protótipo atual e é delineado um roteiro claro para trabalhos futuros, visando a evolução e a completude do Sistema de Gerenciamento de Clínica Médica (SIGM).

5.1 RESULTADOS ALCANÇADOS

O objetivo geral deste trabalho — projetar, implementar e validar um protótipo funcional de um SIGM — foi atingido com sucesso. Os artefatos de Engenharia de Software (Capítulo 3) foram traduzidos com êxito em um sistema web funcional (Capítulo 4), validando a arquitetura MVT (Model-View-Template) do Django como uma escolha robusta para a solução.

Avaliando os objetivos específicos definidos na Seção 1.2.2, constata-se o seguinte:

1. Modelagem de Requisitos e UML: O sistema foi integralmente baseado nos artefatos de projeto. As Histórias de Usuário foram o guia para o desenvolvimento de *features*, enquanto os Diagramas de Casos de Uso e de Classes definiram o escopo e a estrutura de dados, respectivamente. As decisões de projeto (Seção 3.3) demonstram a tradução técnica desses modelos para o código.
2. Implementação da Arquitetura: A arquitetura MVT foi implementada com sucesso. O Diagrama de Classes foi mapeado para o ORM do Django (`models.py`), e os *controllers* conceituais foram implementados como `views.py` coesas e desacopladas, seguindo os padrões GRASP (Controller, Information Expert).
3. Módulos de Autenticação e Perfis: Foram implementados com sucesso. O sistema gerencia três tipos de usuários (`Paciente`, `Médico`, `Recepcionista`) e foi refatorado para utilizar um *backend* de autenticação (`EmailOrUsernameBackend`) que permite o login via e-mail, alinhando-se às práticas de usabilidade modernas.

4. Fluxo de Agendamento: O ciclo de vida do agendamento foi totalmente implementado, incluindo as lógicas de negócio críticas, como a verificação de conflitos de horário (lógica da **Agenda**) e a regra de cancelamento com 24 horas de antecedência.
5. Integração de Pagamentos e Webhooks: Este objetivo complexo foi plenamente alcançado. O sistema é capaz de iniciar uma transação real com o gateway Stripe (**CriarCheckoutSessionView**) e, de forma mais crítica, processar a confirmação de pagamento de maneira assíncrona através de um *endpoint de webhook* seguro (**stripe_webhook_view**). Este resultado valida a capacidade do sistema de lidar com integrações externas complexas.
6. Módulo de Prontuário Eletrônico: O fluxo de atendimento do médico foi concluído. O sistema permite ao médico consultar a agenda, acessar o prontuário histórico completo do paciente e, durante o atendimento, registrar anamnese, emitir receitas e atestados, e, por fim, marcar a consulta como **CONCLUÍDA**.

5.2 TRABALHOS FUTUROS

- Serviços Automatizados (e-mails, lembretes).
- Migração de SGBD (PostgreSQL).
- Deploy em Nuvem (Render/AWS).